

LAPORAN TUGAS
PRAKTIKUM CPU SCHEDULING



Dosen Pengampu =
Triawan Adi Cahyanto M.Kom

Disusun Oleh =
M Ayatullah Mutawakki F (2410651011)

PROGRAM STUDY TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

PENDAHULUAN

A. Tujuan

1. Mempelajari cara kerja algoritma penjadwalan CPU.
2. Membandingkan performa algoritma FCFS, SJF, Dan Round Robin.
3. Menentukan algoritma terbaik berdasarkan AWT, TAT, Dan context switching.

B. Dasar Teori

Penjadwalan CPU adalah proses menentukan urutan eksekusi proses yang berada di antrian ready.

1. FCFS ialah proses dijalankan berdasarkan urutan kedatangan.
2. SJF ialah proses dengan waktu burst terkecil dijalankan terlebih dahulu.
3. Round Robin ialah setiap prosesmendapat jatah waktu.

C. Langkah Praktikum

Tugas 1: Eksperimen dengan data berbeda

Menjalankan program kode cpu sheulding dengan dua set data berbeda dan hasil nya untuk tiga program:

- FCFS
- SJF
- Round Robin

Data yang digunakan:

Set 1

Proses	Arrival Time	Burst Time
P1	0	10
P2	1	5
P3	2	8

Set 2

Proses	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Jalankan kode scheduling.c:

```
#include <stdio.h>
#include <string.h>

struct Process {
    char id[5];
```

```

    int at, bt, ct, tat, wt, remaining;
};

// Fungsi FCFS
void fcfs(struct Process p[], int n) {
    int time = 0;
    float total_wt = 0, total_tat = 0;

    printf("\n===== FCFS Scheduling =====\n");
    for (int i = 0; i < n; i++) {
        if (time < p[i].at)
            time = p[i].at;
        p[i].ct = time + p[i].bt;
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;
        time = p[i].ct;
        total_wt += p[i].wt;
        total_tat += p[i].tat;
    }

    printf("Process\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++)
        printf("%s\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].at, p[i].bt, p[i].ct,
p[i].tat, p[i].wt);

    printf("Average Waiting Time: %.2f\n", total_wt / n);
    printf("Average Turnaround Time: %.2f\n", total_tat / n);
}

// Fungsi SJF Non-preemptive
void sjf(struct Process p[], int n) {
    int completed = 0, time = 0, idx, min_bt;
    float total_wt = 0, total_tat = 0;
    int done[10] = {0};

    printf("\n===== SJF Scheduling =====\n");
    while (completed < n) {
        idx = -1;
        min_bt = 9999;
        for (int i = 0; i < n; i++) {
            if (!done[i] && p[i].at <= time && p[i].bt < min_bt) {
                idx = i;
                min_bt = p[i].bt;
            }
        }
        if (idx == -1) {
            time++;
            continue;
        }
        time += p[idx].bt;

```

```

        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
        done[idx] = 1;
        completed++;
        total_wt += p[idx].wt;
        total_tat += p[idx].tat;
    }

    printf("Process\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++)
        printf("%s\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].at, p[i].bt, p[i].ct,
        p[i].tat, p[i].wt);

    printf("Average Waiting Time: %.2f\n", total_wt / n);
    printf("Average Turnaround Time: %.2f\n", total_tat / n);
}

// Fungsi Round Robin
void rr(struct Process p[], int n, int quantum) {
    int time = 0, completed = 0;
    float total_wt = 0, total_tat = 0;

    printf("\n===== Round Robin (Q=%d) =====\n", quantum);
    while (completed < n) {
        int all_done = 1;
        for (int i = 0; i < n; i++) {
            if (p[i].remaining > 0 && p[i].at <= time) {
                all_done = 0;
                if (p[i].remaining > quantum) {
                    time += quantum;
                    p[i].remaining -= quantum;
                } else {
                    time += p[i].remaining;
                    p[i].remaining = 0;
                    p[i].ct = time;
                    p[i].tat = p[i].ct - p[i].at;
                    p[i].wt = p[i].tat - p[i].bt;
                    total_wt += p[i].wt;
                    total_tat += p[i].tat;
                    completed++;
                }
            }
        }
        if (all_done) time++;
    }

    printf("Process\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++)

```

```

        printf("%s\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].at, p[i].bt, p[i].ct,
p[i].tat, p[i].wt);

        printf("Average Waiting Time: %.2f\n", total_wt / n);
        printf("Average Turnaround Time: %.2f\n", total_tat / n);
    }

int main() {
    int dataset;
    printf("Pilih Data Set (1 / 2): ");
    scanf("%d", &dataset);

    if (dataset == 1) {
        struct Process p1[3] = {
            {"P1",0,10,0,0,10},
            {"P2",1,5,0,0,5},
            {"P3",2,8,0,0,8}
        };
        fcfs(p1,3);
        sjf(p1,3);
        rr(p1,3,2);
    }
    else if (dataset == 2) {
        struct Process p2[4] = {
            {"P1",0,5,0,0,5},
            {"P2",1,3,0,0,3},
            {"P3",2,8,0,0,8},
            {"P4",3,6,0,0,6}
        };
        fcfs(p2,4);
        sjf(p2,4);
        rr(p2,4,2);
    }
    else {
        printf("Pilihan tidak valid!\n");
    }

    return 0;
}

```

Compile dan jalankan:

```

gcc scheduling.c -o scheduling
./scheduling

```

Lalu pilih:

Pilih data set 1 atau 2

Jika 1 hasilnya:

```

./scheduling
Pilih Data Set (1 / 2): 1

===== FCFS Scheduling =====
Process AT      BT      CT      TAT      WT
P1      0      10      10      10      0
P2      1      5      15      14      9
P3      2      8      23      21      13
Average Waiting Time: 7.33
Average Turnaround Time: 15.00

===== SJF Scheduling =====
Process AT      BT      CT      TAT      WT
P1      0      10      10      10      0
P2      1      5      15      14      9
P3      2      8      23      21      13
Average Waiting Time: 7.33
Average Turnaround Time: 15.00

===== Round Robin (Q=2) =====
Process AT      BT      CT      TAT      WT
P1      0      10      23      23      13
P2      1      5      15      14      9
P3      2      8      21      19      11
Average Waiting Time: 11.00
Average Turnaround Time: 18.67

```

Jika 2 hasilnya:

```

Pilih Data Set (1 / 2): 2

===== FCFS Scheduling =====
Process AT      BT      CT      TAT      WT
P1      0      5      5      5      0
P2      1      3      8      7      4
P3      2      8      16     14      6
P4      3      6      22     19     13
Average Waiting Time: 5.75
Average Turnaround Time: 11.25

===== SJF Scheduling =====
Process AT      BT      CT      TAT      WT
P1      0      5      5      5      0
P2      1      3      8      7      4
P3      2      8      22     20     12
P4      3      6      14     11      5
Average Waiting Time: 5.25
Average Turnaround Time: 10.75

===== Round Robin (Q=2) =====
Process AT      BT      CT      TAT      WT
P1      0      5      16     16     11
P2      1      3      11     10      7
P3      2      8      22     20     12
P4      3      6      20     17     11
Average Waiting Time: 10.25
Average Turnaround Time: 15.75

```

Tugas 2: Analisis hasil eksperimen dari ketiga algoritma

1. Algoritma average waiting time terkecil ialah SJF

2. Mengapa hasilnya berbeda?

Karena strategi pemilihan nya berbeda. Seperti FCFS berdasarkan urutan datang, SJF berdasarkan durasi, RR berdasarkan giliran waktu.

3. Kapan sebaiknya menggunakan FCFS?

Ketika sistem batch dengan beban kerja seragam dan tanpa interaksi pengguna.

4. Kapan sebaiknya menggunakan FCFS?

Cocok untuk sistem yang memprioritaskan efisiensi CPU dan waktu tunggu rendah.

5. Kapan sebaiknya menggunakan Round Robin?

Round Robin sangat cocok untuk sistem operasi multitasking, di mana banyak proses berjalan secara bersamaan. Algoritma ini memastikan setiap proses mendapatkan waktu eksekusi yang adil dengan membagi waktu secara bergiliran.

Tugas 3: Modifikasi Program CPU Scheduling Round Robin (Quantum = 2)

Isi file input (input.txt.):

Contoh file yang akan dibaca:

3		
P1	0	5
P2	1	3
P3	2	8

Maksudnya ada 3 proses, dan format tiap baris: NamaProses ArrivalTime BurstTime.

Untuk Kode program nya:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Process {
    char name[5];
    int arrival, burst, remaining, waiting, turnaround;
};

int main() {
    struct Process p[10];
    int n, quantum = 2;
    FILE *input, *output;
    char gantt[100] = "";

    // Baca data dari file input.txt
    input = fopen("input.txt", "r");
    if (input == NULL) {
        printf("File input.txt tidak ditemukan!\n");
        return 1;
    }

    fscanf(input, "%d", &n);
    for (int i = 0; i < n; i++) {
```



```

    fscanf(input, "%s %d %d", p[i].name, &p[i].arrival, &p[i].burst);
    p[i].remaining = p[i].burst;
    p[i].waiting = 0;
    p[i].turnaround = 0;
}
fclose(input);

// Algoritma Round Robin
int time = 0, done = 0;
while (done < n) {
    int executed = 0;
    for (int i = 0; i < n; i++) {
        if (p[i].arrival <= time && p[i].remaining > 0) {
            executed = 1;
            int run = (p[i].remaining > quantum) ? quantum : p[i].remaining;
            p[i].remaining -= run;
            time += run;

            char buf[10];
            sprintf(buf, "%s(%d)", p[i].name, time);
            strcat(gantt, buf);

            if (p[i].remaining == 0) {
                done++;
                p[i].turnaround = time - p[i].arrival;
                p[i].waiting = p[i].turnaround - p[i].burst;
            }
        }
    }
    if (!executed)
        time++;
}

```

```

// Hitung rata-rata
float avgWait = 0, avgTurn = 0;
for (int i = 0; i < n; i++) {
    avgWait += p[i].waiting;
    avgTurn += p[i].turnaround;
}
avgWait /= n;
avgTurn /= n;

// Simpan hasil ke file output.txt
output = fopen("output.txt", "w");
fprintf(output, "=== Hasil CPU Scheduling (Round Robin) ===\n\n");
fprintf(output, "Nama\tArrival\tBurst\tWaiting\tTurnaround\n");
for (int i = 0; i < n; i++) {
    fprintf(output, "%s\t%d\t%d\t%d\t%d\n", p[i].name, p[i].arrival,
p[i].burst, p[i].waiting, p[i].turnaround);
}
fprintf(output, "\nRata-rata Waiting Time: %.2f\n", avgWait);
fprintf(output, "Rata-rata Turnaround Time: %.2f\n", avgTurn);
fprintf(output, "\nGantt Chart:\n%s\n", gantt);
fclose(output);

printf("Hasil simulasi berhasil disimpan ke output.txt!\n");
return 0;
}

```

Artinya :

1. Input dari file program membaca jumlah prosesnya dan detailnya dari file.
2. Proses raoun robin: setiap proses mendapatkan waktu eksekusi maksimal sebesar quantum yakni 2 satuan waktu. Jika belum selesai, proses akan mendapatkan giliran lagi di putaran berikutnya.
3. Giant Chart dibentuk saat proses dijalankan, misalnya:

|P1(2)|P2(4)|P3(6)|P1(8)|P3(10)|P3(12)|

4. Hasil compile dan Output ke file:

```
aqil@DESKTOP-RGGJ3FG:~$ ./modifikasi
✓ Hasil simulasi berhasil disimpan ke output.txt!
aqil@DESKTOP-RGGJ3FG:~$
```

Untuk melihat hasilnya pakai:

cat output.txt

```
aqil@DESKTOP-RGGJ3FG:~$ cat output.txt
=== Hasil CPU Scheduling (Round Robin) ===

Nama      Arrival Burst   Waiting Turnaround
P1         0         5         7         12
P2         1         3         5          8
P3         2         8         6         14

Rata-rata Waiting Time: 6.00
Rata-rata Turnaround Time: 11.33

Gantt Chart:
|P1(2)|P2(4)|P3(6)|P1(8)|P2(9)|P3(11)|P1(12)|P3(14)|P3(16)|
```

Di sini setelah ditambahkan fitur baca dan tulis file, program menjadi lebih interaktif dan efisien. Karena gantt chart detail membantu memahami urutan eksekusi proses lebih visual.

Rata-rata waktu tunggu meningkat karena setiap proses menjadi giliran, namun tidak ada proses yang terabaikan.

Tugas 4: Eksperimen dengan data berbeda untuk perbandingan algoritma

Di bawah ini hasil eksperimen dari 3 algoritma (FCFS, SJF, RR) menggunakan 3 test case.

Hasilnya dihitung dengan rumus umum:

Turnaround Time (TAT) = Finish Time – Arrival Time

Waiting Time (WT) = TAT – Burst Time

Langkanya:

Buat file: tugas4.c

Buat kodenya:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 10

typedef struct {
    char name[5];
    int arrival, burst, remaining, waiting, turnaround, completion;
} Process;

void fcfs(Process p[], int n);
void sjf(Process p[], int n);
void rr(Process p[], int n, int tq);
void print_result(Process p[], int n, const char *algo);

int main() {
    FILE *in = fopen("input.txt", "r");
    if (!in) {
        printf("✗ Tidak dapat membuka file input.txt\n");
        return 1;
    }

    int n, tq;
```

```

fscanf(in, "%d", &n);
Process p[MAX];
for (int i = 0; i < n; i++) {
    fscanf(in, "%s %d %d", p[i].name, &p[i].arrival, &p[i].burst);
    p[i].remaining = p[i].burst;
}
fscanf(in, "%d", &tq); // time quantum (jika ada)
fclose(in);

printf("Data proses berhasil dibaca dari input.txt ✅\n\n");

fcfs(p, n);
sjf(p, n);
rr(p, n, tq);
return 0;
}

// ----- FCFS -----
void fcfs(Process p[], int n) {
    int time = 0;
    Process temp[MAX];
    memcpy(temp, p, sizeof(Process)*n);

    printf("=== FCFS Scheduling ===\nGantt Chart: ");
    for (int i = 0; i < n; i++) {
        if (time < temp[i].arrival) time = temp[i].arrival;
        printf("| %s ", temp[i].name);
        time += temp[i].burst;
        temp[i].completion = time;
        temp[i].turnaround = temp[i].completion - temp[i].arrival;
        temp[i].waiting = temp[i].turnaround - temp[i].burst;
    }
    printf("\n");
}

```

```

    print_result(temp, n, "FCFS");
}

// ----- SJF -----
void sjf(Process p[], int n) {
    Process temp[MAX];
    memcpy(temp, p, sizeof(Process)*n);
    int completed = 0, time = 0, min_idx;
    int done[MAX] = {0};

    printf("\n==== SJF Scheduling ==== \nGantt Chart: ");
    while (completed < n) {
        min_idx = -1;
        for (int i = 0; i < n; i++) {
            if (temp[i].arrival <= time && !done[i]) {
                if (min_idx == -1 || temp[i].burst < temp[min_idx].burst)
                    min_idx = i;
            }
        }
        if (min_idx == -1) {
            time++;
            continue;
        }
        printf("| %s ", temp[min_idx].name);
        time += temp[min_idx].burst;
        temp[min_idx].completion = time;
        temp[min_idx].turnaround = time - temp[min_idx].arrival;
        temp[min_idx].waiting = temp[min_idx].turnaround -
temp[min_idx].burst;
        done[min_idx] = 1;
        completed++;
    }
    printf("\n");
}

```

```

    print_result(temp, n, "SJF");
}

// ----- Round Robin -----
void rr(Process p[], int n, int tq) {
    Process temp[MAX];
    memcpy(temp, p, sizeof(Process)*n);
    int time = 0, remain = n;

    printf("\n=== Round Robin Scheduling (TQ=%d) ===\nGantt
Chart: ", tq);
    while (remain > 0) {
        int done = 0;
        for (int i = 0; i < n; i++) {
            if (temp[i].arrival <= time && temp[i].remaining > 0) {
                done = 1;
                printf("| %s ", temp[i].name);
                if (temp[i].remaining <= tq) {
                    time += temp[i].remaining;
                    temp[i].remaining = 0;
                    temp[i].completion = time;
                    temp[i].turnaround = temp[i].completion -
temp[i].arrival;
                    temp[i].waiting = temp[i].turnaround - temp[i].burst;
                    remain--;
                } else {
                    temp[i].remaining -= tq;
                    time += tq;
                }
            }
        }
        if (!done) time++;
    }
}

```

```

printf("\n");
print_result(temp, n, "Round Robin");
}

// ----- Cetak hasil ke terminal & file -----
void print_result(Process p[], int n, const char *algo) {
    FILE *out = fopen("output.txt", "a");
    float avg_wt = 0, avg_tat = 0;

    fprintf(out, "\n=== %s ===\n", algo);
    printf("-----\n");
    printf("| %s | %s | %s | %s | %s |\n", "Proses", "AT", "BT", "WT",
    "TAT");
    printf("-----\n");

    for (int i = 0; i < n; i++) {
        avg_wt += p[i].waiting;
        avg_tat += p[i].turnaround;
        printf("| %-6s | %-2d | %-2d | %-2d | %-2d |\n",
            p[i].name,    p[i].arrival,    p[i].burst,    p[i].waiting,
            p[i].turnaround);
        fprintf(out, "%s %d %d %d %d\n",
            p[i].name,    p[i].arrival,    p[i].burst,    p[i].waiting,
            p[i].turnaround);
    }
    avg_wt /= n;
    avg_tat /= n;
    printf("-----\n");
    printf("Rata-rata WT = %.2f, Rata-rata TAT = %.2f\n", avg_wt,
    avg_tat);
    fprintf(out, "Rata-rata WT = %.2f, Rata-rata TAT = %.2f\n",
    avg_wt, avg_tat);
    fclose(out);
}

```



```
}
```

Buat file input:

nano input.txt

contoh test case 1:

```
GNU nano 7.2
4
P1 0 24
P2 1 3
P3 2 3
P4 3 6
3
```

Jalankan:

gcc tugas4.c -o tugas4

./tugas4

```
Data proses berhasil dibaca dari input.txt ✓

=== FCFS Scheduling ===
Gantt Chart: | P1 | P2 | P3 | P4 |
-----
| Proses | AT | BT | WT | TAT |
-----
| P1     | 0  | 24 | 0  | 24 |
| P2     | 1  | 3  | 23 | 26 |
| P3     | 2  | 3  | 25 | 28 |
| P4     | 3  | 6  | 27 | 33 |
-----
Rata-rata WT = 18.75, Rata-rata TAT = 27.75

=== SJF Scheduling ===
Gantt Chart: | P1 | P2 | P3 | P4 |
-----
| Proses | AT | BT | WT | TAT |
-----
| P1     | 0  | 24 | 0  | 24 |
| P2     | 1  | 3  | 23 | 26 |
| P3     | 2  | 3  | 25 | 28 |
| P4     | 3  | 6  | 27 | 33 |
-----
Rata-rata WT = 18.75, Rata-rata TAT = 27.75

=== Round Robin Scheduling (TQ=3) ===
Gantt Chart: | P1 | P2 | P3 | P4 | P1 | P4 | P1 | P1 | P1 | P1 | P1 | P1 |
-----
| Proses | AT | BT | WT | TAT |
-----
| P1     | 0  | 24 | 12 | 36 |
| P2     | 1  | 3  | 2  | 5  |
| P3     | 2  | 3  | 4  | 7  |
| P4     | 3  | 6  | 9  | 15 |
-----
Rata-rata WT = 6.75, Rata-rata TAT = 15.75
```

Hasilnya akan tampil seperti ini dan juga otomatis disimpan di file output.txt

Untuk test case 2 dan 3 ulangi langkahnya seperti pada Langkah test case 1, yakni tinggal ubah input.txt sesuai test case:

Test case 2:

5

P1 0 10

P2 0 5

P3 0 8

P4 0 12

P5 0 6

4

Hasilnya:

```
aqil@DESKTOP-RGGJ3FG:~$ ./tugas4
Data proses berhasil dibaca dari input.txt ✓

=== FCFS Scheduling ===
Gantt Chart: | P1 | P2 | P3 | P4 | P5 |
-----
| Proses | AT | BT | WT | TAT |
-----
| P1      | 0  | 10 | 0  | 10  |
| P2      | 0  | 5  | 10 | 15  |
| P3      | 0  | 8  | 15 | 23  |
| P4      | 0  | 12 | 23 | 35  |
| P5      | 0  | 6  | 35 | 41  |
-----
Rata-rata WT = 16.60, Rata-rata TAT = 24.80

=== SJF Scheduling ===
Gantt Chart: | P2 | P5 | P3 | P1 | P4 |
-----
| Proses | AT | BT | WT | TAT |
-----
| P1      | 0  | 10 | 19 | 29  |
| P2      | 0  | 5  | 0  | 5   |
| P3      | 0  | 8  | 11 | 19  |
| P4      | 0  | 12 | 29 | 41  |
| P5      | 0  | 6  | 5  | 11  |
-----
Rata-rata WT = 12.80, Rata-rata TAT = 21.00

=== Round Robin Scheduling (TQ=4) ===
Gantt Chart: | P1 | P2 | P3 | P4 | P5 | P1 | P2 | P3 | P4 | P5 | P1 | P4 |
-----
| Proses | AT | BT | WT | TAT |
-----
| P1      | 0  | 10 | 27 | 37  |
| P2      | 0  | 5  | 20 | 25  |
| P3      | 0  | 8  | 21 | 29  |
| P4      | 0  | 12 | 29 | 41  |
| P5      | 0  | 6  | 29 | 35  |
-----
Rata-rata WT = 25.20, Rata-rata TAT = 33.40
```

Test case 3:

5

P1 0 8

P2 2 4

P3 4 9

P4 6 5

P5 8 3

2

Hasilnya:

```
aqil@DESKTOP-RGGJ3FG:~$ ./tugas4
Data proses berhasil dibaca dari input.txt ✓

=== FCFS Scheduling ===
Gantt Chart: | P1 | P2 | P3 | P4 | P5 |
-----
| Proses | AT | BT | WT | TAT |
-----
| P1      | 0  | 8  | 0  | 8  |
| P2      | 2  | 4  | 6  | 10 |
| P3      | 4  | 9  | 8  | 17 |
| P4      | 6  | 5  | 15 | 20 |
| P5      | 8  | 3  | 18 | 21 |
-----
Rata-rata WT = 9.40, Rata-rata TAT = 15.20

=== SJF Scheduling ===
Gantt Chart: | P1 | P5 | P2 | P4 | P3 |
-----
| Proses | AT | BT | WT | TAT |
-----
| P1      | 0  | 8  | 0  | 8  |
| P2      | 2  | 4  | 9  | 13 |
| P3      | 4  | 9  | 16 | 25 |
| P4      | 6  | 5  | 9  | 14 |
| P5      | 8  | 3  | 0  | 3  |
-----
Rata-rata WT = 6.80, Rata-rata TAT = 12.60

=== Round Robin Scheduling (TQ=2) ===
Gantt Chart: | P1 | P2 | P3 | P4 | P5 | P1 | P2 | P3 | P4 | P5 | P1 | P3 | P4 | P1 | P3 | P3 |
-----
| Proses | AT | BT | WT | TAT |
-----
| P1      | 0  | 8  | 18 | 26 |
| P2      | 2  | 4  | 8  | 12 |
| P3      | 4  | 9  | 16 | 25 |
| P4      | 6  | 5  | 13 | 18 |
| P5      | 8  | 3  | 8  | 11 |
-----
Rata-rata WT = 12.60, Rata-rata TAT = 18.40
```

- Dari hasil test case tiga tersebut SJF memberikan hasil paling efisien yakni WT dan TAT kecil.
- Round Robin cocok untuk semua interaktif.
- FCFS cocok untuk system batch tanpa interaktif.

Penjelasan untuk test case, misal:

4

P1 0 24

P2 1 3

P3 2 3

P4 3 6

3

- 4 = jumlah proses
- P1 – P4 = nama proses
- Angka kedua = Arrival Time
- Angka ketiga = Burst Time

- Baris terakhir 3 = Time Quantum (khusus round robin)

Tugas 5 Analisis Eksperimen Perbandingan Algoritma CPU Scheduling

1. Analisis Performa

- a. Dalam kondisi apa FCFS memberikan hasil terbaik?

FCFS bagus kalau semua proses datangnya bareng dan waktu eksekusinya hampir sama. Soalnya dia ngerjain dari yang datang duluan sampai habis, jadi kalau datanya seimbang hasilnya juga rata.

- b. Kenapa SJF hamper selalu paling kecil Average Waiting Timenya?

Karena ngerjain prosesnya yang lebih cepat dulu.

Karena proses kecil diselesaikan dulu, total waktu tunggu semua prosesnya jadi lebih kecil. Makanya rata-rata waktunya paling kecil dibandingkan algoritma lainnya.

- c. Kekurangan trade off dari Round Robin

Round robin ini adil, karena semua proses dapat giliran. Akan tetapi, karena sering ganti proses, CPU bisa sibuk Cuma buat ganti-ganti, bukan ngerjain perintah tugasnya.

Kalau time quantum terlalu kecil, ganti-gantinya kebanyakan.

Kalau terlalu besar, malah mirip FCFS lagi.

2. Context Switching

waktu CPU pindah dari satu proses ke proses lain. Semakin sering pindah, semakin besar waktu yang kebuang buat nyimpen dan balikin data proses sebelumnya.

Jika context switching terlalu sering, sistem bisa jadi lebih lambat.

Tapi kalau jarang, sistem bisa gak responsif. Jadi harus seimbang.

KESIMPULAN

Dari semua percobaan yang udah dilakukan, bisa disimpulkan kalau setiap algoritma CPU scheduling punya kelebihan dan kekurangan masing-masing.

- FCFS paling sederhana, cocok untuk sistem yang prosesnya datang bersamaan dan tidak butuh respon cepat.
- SJF paling efisien karena waktu tungguanya paling kecil, tapi perlu tahu waktu eksekusi proses terlebih dahulu.
- Round Robin paling adil karena semua proses dapat giliran, tapi sering terjadi context switching yang bikin waktu CPU sedikit terbuang.

Secara umum, tidak ada algoritma yang paling sempurna untuk semua kondisi. Pemilihan algoritma tergantung pada tujuan sistemnya apakah ingin efisien, adil, atau cepat merespon proses pengguna.